JavaScript is disabled on your browser.

compact1, compact2, compact3

java.util.concurrent

Interface CompletionService<V>

All Known Implementing Classes:

ExecutorCompletionService

---

public interface **CompletionService<V>**

A service that decouples the production of new asynchronous tasks from the consumption of the results of completed tasks. Producers `submit` tasks for execution. Consumers `take` completed tasks and process their results in the order they complete. A `CompletionService` can for example be used to manage asynchronous I/O, in which tasks that perform reads are submitted in one part of a program or system, and then acted upon in a different part of the program when the reads complete, possibly in a different order than they were requested.

Typically, a `CompletionService` relies on a separate `Executor` to actually execute the tasks, in which case the `CompletionService` only manages an internal completion queue. The `ExecutorCompletionService` class provides an implementation of this approach.

Memory consistency effects: Actions in a thread prior to submitting a task to a `CompletionService` *happen-before* actions taken by that task, which in turn *happen-before* actions following a successful return from the corresponding `take()`.

Method Summary

Abstract Methods

Modifier and Type	Method	Description
Future<V>	**poll()**	

		Retrieves and removes the Future representing the next completed task, or `null` if none are present.
Future<**V**>	**poll**(long timeout, **TimeUnit** unit)	Retrieves and removes the Future representing the next completed task, waiting if necessary up to the specified wait time if none are yet present.
Future<**V**>	**submit**(**Callable**<**V**> task)	Submits a value-returning task for execution and returns a Future representing the pending results of the task.
Future<**V**>	**submit**(**Runnable** task, **V** result)	Submits a Runnable task for execution and returns a Future representing that task.
Future<**V**>	**take**()	Retrieves and removes the Future representing the next completed task, waiting if none are yet present.

Method Detail

submit

`Future<V> submit(Callable<V> task)`

Submits a value-returning task for execution and returns a Future representing the pending results of the task. Upon completion, this task may be taken or polled.

Parameters:

`task - the task to submit`

Returns:

`a Future representing pending completion of the task`

Throws:

`RejectedExecutionException - if the task cannot be scheduled for execution`

NullPointerException - if the task is null

submit

```
Future<V> submit(Runnable task,
                 V result)
```

Submits a Runnable task for execution and returns a Future representing that task. Upon completion, this task may be taken or polled.

Parameters:

task - the task to submit

result - the result to return upon successful completion

Returns:

a Future representing pending completion of the task, and whose get() method will return the given result value upon completion

Throws:

RejectedExecutionException - if the task cannot be scheduled for execution

NullPointerException - if the task is null

take

```
Future<V> take()
        throws InterruptedException
```

Retrieves and removes the Future representing the next completed task, waiting if none are yet present.

Returns:

the Future representing the next completed task

Throws:

InterruptedException - if interrupted while waiting

poll

```
Future<V> poll()
```

Retrieves and removes the Future representing the next completed task, or `null` if none are present.

Returns:

the Future representing the next completed task, or null if none are present

poll

```
Future<V> poll(long timeout,
               TimeUnit unit)
        throws InterruptedException
```

Retrieves and removes the Future representing the next completed task, waiting if necessary up to the specified wait time if none are yet present.

Parameters:

timeout - how long to wait before giving up, in units of unit

unit - a TimeUnit determining how to interpret the timeout parameter

Returns:

the Future representing the next completed task or null if the specified waiting time elapses before one is present

Throws:

InterruptedException - if interrupted while waiting

workarounds, and working code examples.
Copyright © 1993, 2026, Oracle and/or its affiliates. All rights reserved. Use is subject to license terms.
Also see the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.